

HTML Styles CSS

(Cascading Style Sheets)

HTML Styles – CSS

CSS (Cascading Style Sheets) is used to style HTML elements

What is CSS?

- **CSS** stands for **Cascading Style Sheets**
- Not a programming language
- Used for website Layout and Design.
- Styles define **how to display** HTML elements i.e. used for styling.
- Styles were added to HTML 4.0 **to solve a problem**
- **External Style Sheets** can save a lot of work
- External Style Sheets are stored in **CSS files**

Styles Solved a Big Problem

HTML was never intended to contain tags for formatting a document.

HTML was intended to define the content of a document, like:

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

When tags like , and color attributes were added to the HTML 3.2 specification, it started a nightmare for web developers. Development of large web sites, where fonts and color information were added to every single page, became a long and expensive process.

To solve this problem, the World Wide Web Consortium (W3C) created CSS.

In HTML 4.0 and onwards, all formatting could be removed from the HTML document, and stored in a separate CSS file.

What We Need To Start

- A Web Browser (*Pick One*)
 - Google Chrome
 - Mozilla Firefox
 - Safari
 - Edge
 - IE (Please Don't)
- A Text Editor (*Pick One*)
 - Sublime Text
 - Atom.io
 - Visual Studio Code
 - Brackets
 - Notepad++ (Windows)
 - TextMate (Mac)

CSS Saves a Lot of Work!

- CSS defines HOW HTML elements are to be displayed.
- Styles are normally saved in external .css files. External style sheets enable you to change the appearance and layout of all the pages in a Web site, just by editing one single file!

Styling HTML with CSS

CSS was introduced together with HTML 4, to provide a better way to style HTML elements.

CSS can be added to HTML in the following 3 ways:

- Inline - using the style **attribute** in HTML elements
- Internal - using the <style> **element** in the <head> section
- External - using an external CSS **file**

The preferred way to add CSS to HTML, is to put CSS syntax in separate CSS files.

Inline Styles

An inline style loses many of the advantages of style sheets by mixing content with presentation. Use this method sparingly!

To use inline styles you use the style attribute in the relevant tag. The style attribute can contain any CSS property. The example shows how to change the color and the left margin of a paragraph:

```
<p style="color:sienna;margin-left:20px;">This is a paragraph</p>
```

Internal Style Sheet

An internal style sheet should be used when a single document has a unique style. You define internal styles in the head section of an HTML page, by using the <style> tag, like this:

```
<head>
```

```
<style>
```

```
hr {color:sienna;}
```

```
p {margin-left:20px;}
```

```
body {background-image:url("images/back40.gif");}
```

```
</style>
```

```
</head>
```

External Style Sheet

An external style sheet is ideal when the style is applied to many pages. With an external style sheet, you can change the look of an entire Web site by changing one file. Each page must link to the style sheet using the <link> tag. The <link> tag goes inside the head section:

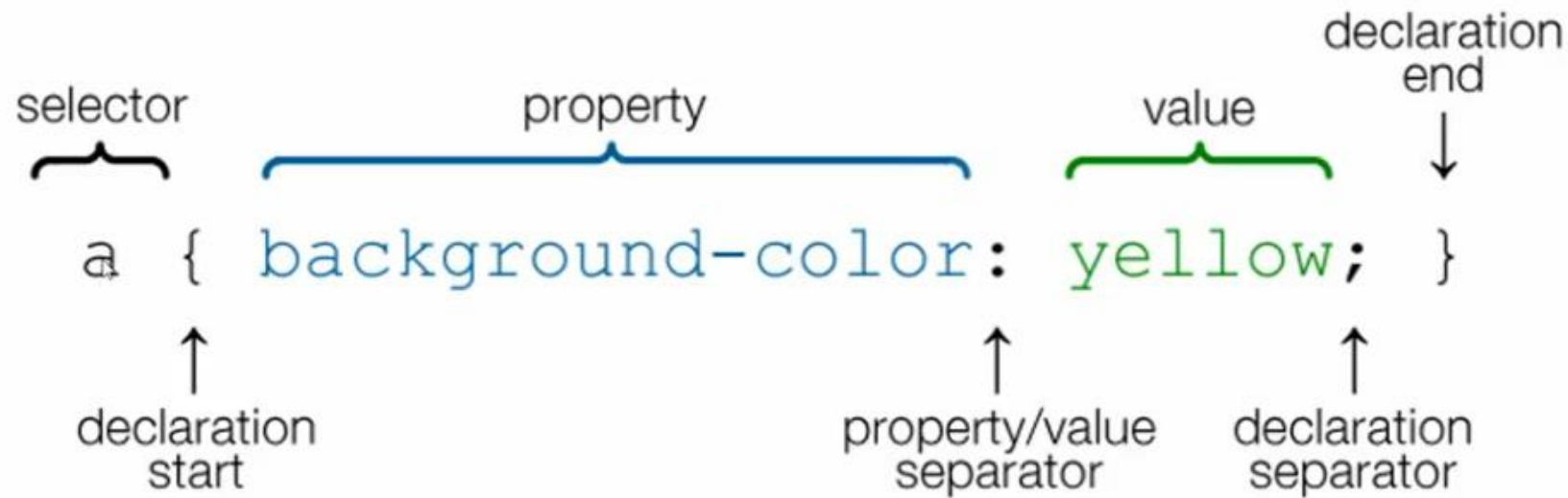
```
<head>  
  <link rel="stylesheet" type="text/css" href="mystyle.css">  
</head>
```

An external style sheet can be written in any text editor. The file should not contain any html tags. Your style sheet should be saved with a .css extension.

Do not add a space between the property value and the unit (such as margin-left:20 px). The correct way is: margin-left:20px

```
hr {color:sienna;}  
p {margin-left:20px;}  
  
body {background-image:url("images/back40.gif");}
```

CSS Selectors



CSS Syntax

A CSS rule has two main parts: a selector, and one or more declarations:



The selector is normally the HTML element you want to style.

Each declaration consists of a property and a value. The property is the style attribute you want to change. Each property has a value.

CSS Comments

Comments are used to explain your code, and may help you when you edit the source code at a later date. Comments are ignored by browsers.

A CSS comment begins with "/*", and ends with "*/", like this:

```
/*This is a comment*/  
p  
{  
    text-align:center;  
    /*This is another comment*/  
    color:black;  
    font-family:arial;  
}
```

CSS Selector

CSS selectors are used *to select the content you want to style*. Selectors are the part of CSS rule set. CSS selectors select HTML elements according to its id, class, type, attribute etc.

There are several different types of selectors in CSS.

- 1) CSS Element Selector
- 2) CSS Id Selector
- 3) CSS Class Selector
- 4) CSS Universal Selector
- 5) CSS Group Selector

1) CSS Element Selector

The element selector selects the HTML element by name.

```
<!DOCTYPE html>
<html>
<head>
<style>
p{
    color:red;
    text-align:center;
}
</style>
</head>

<body>
<p>Hello World!</p>
<p>This paragraph is
styled with CSS.</p>
</body>
</html>
```

Hello World!
This paragraph is styled with CSS.

2) CSS Id Selector

The id selector selects the id attribute of an HTML element to select a specific element. An id is always unique within the page so it is chosen to select a single, unique element. It is written with the hash character (#), followed by the id of the element.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
#para1
```

```
{
```

```
text-align:center;
```

```
color:red;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p id="para1">Hello World!</p>
```

```
<p>This paragraph is not affected
```

```
by the style.</p>
```

```
</body>
```

```
</html>
```

Hello World!

This paragraph is not affected by the style.

3) CSS class Selector

- The class selector is used to specify a style for a group of elements. Unlike the id selector, the class selector is most often used on several elements.
- This allows you to set a particular style for many HTML elements with the same class.
- The class selector uses the HTML class attribute, and is defined with a "."
- In the example below, all HTML elements with class="center" will be center-aligned:
- Do **NOT** start a class name with a number!

```
<!DOCTYPE html>
<html>
<head>
<style>
.center
{
    text-align:center;
}
</style>
</head>

<body>
<h1 class="center">Center-aligned heading</h1>
<p class="center">Center-aligned paragraph</p>
</body>
</html>
```

Center-aligned heading

Center-aligned paragraph.

4) CSS Universal Selector

The universal selector is used as a wildcard character. It selects all the elements on the pages.

```
<!DOCTYPE html>
<html>
<head>
<style>
* {
  color: green;
  font-size: 20px;
}
</style>
</head>
<body>
<h2>This is heading</h2>
<p>This style will be applied on every paragraph.</p>
<p id="para1">Me too!</p>
<p>And me!</p>
</body>
</html>
```

This is heading

This style will be applied on every paragraph.

Me too!

And me!

5) CSS Group Selector

- The grouping selector is used to select all the elements with the same style definitions.
- Grouping selector is used to minimize the code. Commas are used to separate each selector in grouping.

```
h1 {  
  text-align: center;  
  color: blue;  
}
```

```
h2 {  
  text-align: center;  
  color: blue;  
}
```

```
p {  
  text-align: center;  
  color: blue;  
}
```

As you can see, you need to define CSS properties for all the elements. It can be grouped in following ways:

```
h1,h2,p {  
  text-align: center;  
  color: blue;  
}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
h1, h2, p {  
  text-align: center;  
  color: blue;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Hello Manisha Systems</h1>
```

```
<h2>Hello Manisha Systems  
(In smaller font)</h2>
```

```
<p>This is a paragraph.</p>
```

```
</body>
```

```
</html>
```

Hello Manisha Systems

Hello ManishaSystems (In smaller font)

This is a paragraph

Important css properties

- Font
- Color
- Background Color
- Background
- Box Model (Margin/Padding)
- Border
- Border Radius
- Height/Width.
- Pseudo Elements
- Display
- Position
- Cursor:Pointer
- :hover
- Flex
- Transform
- Tansition
- CSS Variable



CSS Fonts

- Using a font that is easy to read is important. The font adds value to your text.
- It is also important to choose the correct color and text size for the font.
- The right font can create a strong identity for your brand.

Generic Font Families

In CSS there are **five** generic font families:

1. **Serif** fonts have a small stroke at the edges of each letter. They create a sense of formality and elegance.
2. **Sans-serif** fonts have clean lines (no small strokes attached). They create a modern and minimalistic look.
3. **Monospace** fonts - here all the letters have the same fixed width. They create a mechanical look.
4. **Cursive** fonts imitate human handwriting.
5. **Fantasy** fonts are decorative/playful fonts.

Generic Font Family	Examples of Font Names
Serif	Times New Roman Georgia Garamond
Sans-serif	Arial Verdana Helvetica
Monospace	Courier New Lucida Console Monaco
Cursive	<i>Brush Script MT</i> <i>Lucida Handwriting</i>
Fantasy	Copperplate Papyrus

Web Safe Fonts



Examples	Font Families
font-family	Arial, Helvetica, sans-serif
font-family	"Arial Black", Gadget, sans-serif
font-family	"Bookman Old Style", serif
font-family	"Comic Sans MS", cursive, sans-serif
font-family	Courier, monospace
font-family	"Courier New", Courier, monospace
font-family	Garamond, serif
font-family	Georgia, serif
font-family	Impact, Charcoal, sans-serif
font-family	"Lucida Console", Monaco, monospace
font-family	"Lucida Sans Unicode", "Lucida Grande", sans-serif
font-family	"MS Sans Serif", Geneva, sans-serif
font-family	"MS Serif", "New York", sans-serif
font-family	"Palatino Linotype", "Book Antiqua", Palatino, serif
font-family	Symbol, sans-serif
font-family	Tahoma, Geneva, sans-serif
font-family	"Times New Roman", Times, serif
font-family	"Trebuchet MS", Helvetica, sans-serif
font-family	Verdana, Geneva, sans-serif

Colors In CSS

- The standard color system available in modern computers supports 24-bit colors, which allows the display of about 16.7 million distinct colors via a combination of different red, green and blue channels with 256 different values per channel ($256 \times 256 \times 256 = 16,777,216$).
- The color property in CSS is used to set the color of HTML elements. Typically, this property is used to set the background color or the font color of an element.
- In CSS, we use color values for specifying the color. We can also use this property for the border-color and other decorative effects.

We can define the color of an element by using the following ways:

- Built-in color
- RGB format
- RGBA format
- Hexadecimal notation
- HSL
- HSLA



Color keywords (Built-in color)

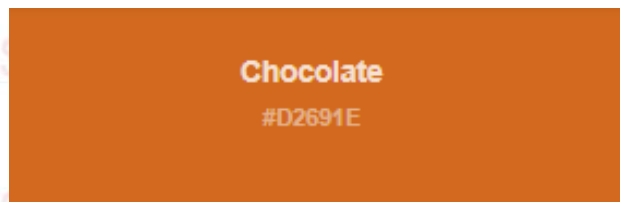
Color keywords are a simple and understandable way of specifying color.

Eg: Red, Green, Yellow, Blue, Black, White, Coral, Crimson, Chocolate, Cornflowerblue, greenyellow etc

`background-color: blue;`

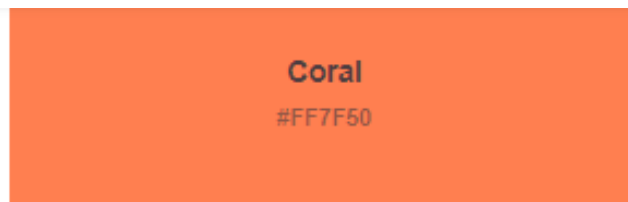
Color	HEX	RGB	Color
	#FF0000	rgb(255,0,0)	Red
	#00FF00	rgb(0,255,0)	Green
	#0000FF	rgb(0,0,255)	Blue

Color	HEX	RGB	Color
	#000000	rgb(0,0,0)	Black
	#808080	rgb(128,128,128)	Gray
	#FFFFFF	rgb(255,255,255)	White



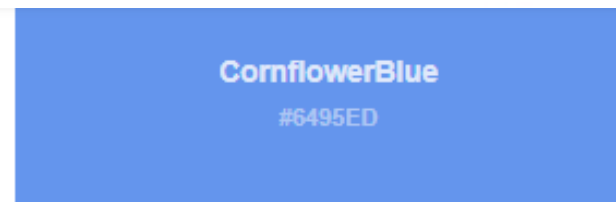
Chocolate

#D2691E



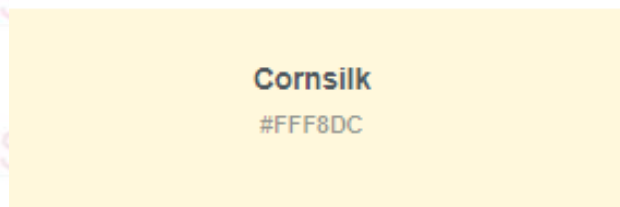
Coral

#FF7F50



CornflowerBlue

#6495ED



Cornsilk

#FFF8DC



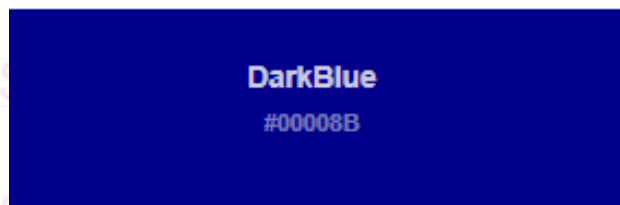
Crimson

#DC143C



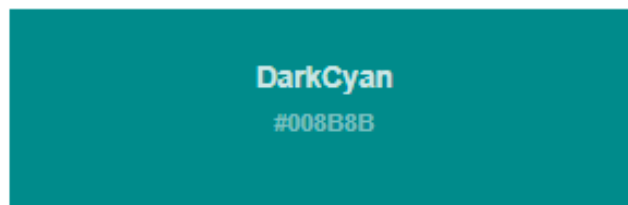
Cyan

#00FFFF



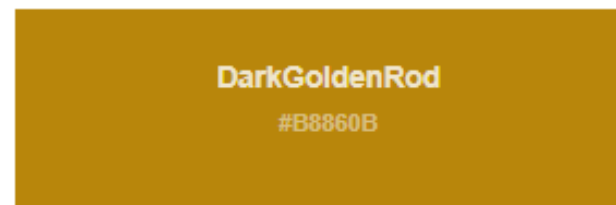
DarkBlue

#00008B



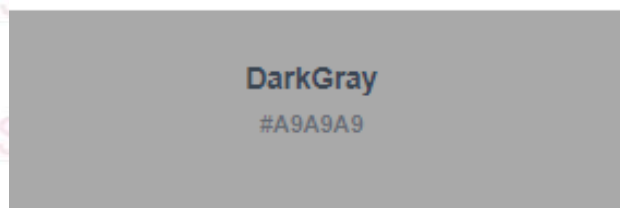
DarkCyan

#008B8B



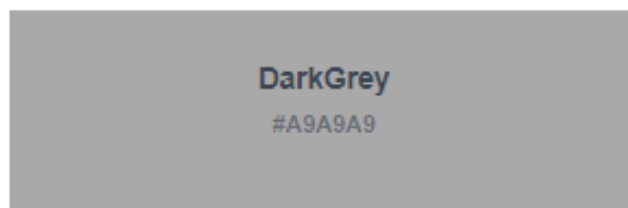
DarkGoldenRod

#B8860B



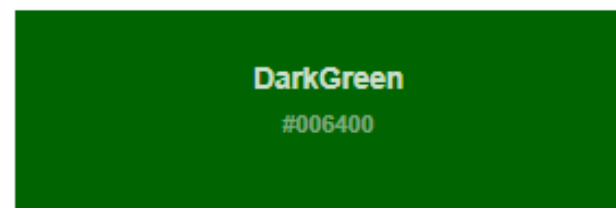
DarkGray

#A9A9A9



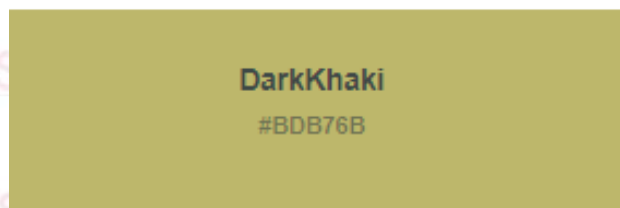
DarkGrey

#A9A9A9



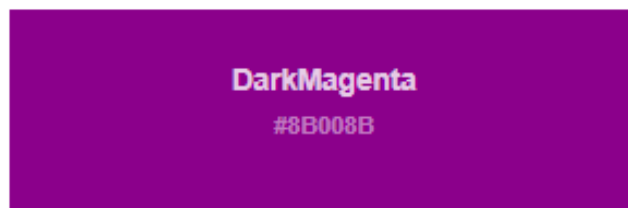
DarkGreen

#006400



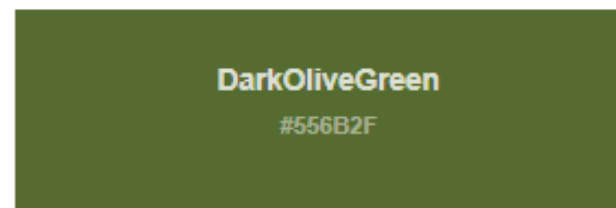
DarkKhaki

#BDB76B



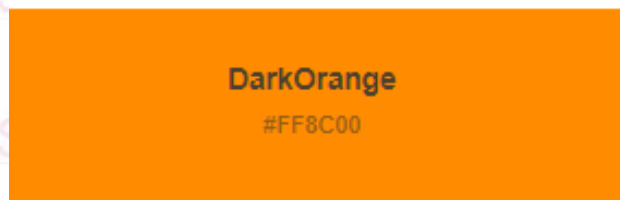
DarkMagenta

#8B008B



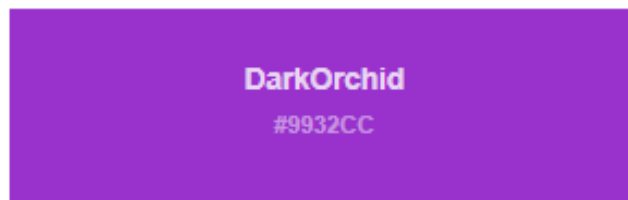
DarkOliveGreen

#556B2F



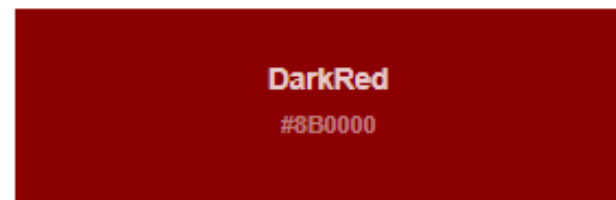
DarkOrange

#FF8C00



DarkOrchid

#9932CC



DarkRed

#8B0000

RGB and RGBA values

- An RGB value is a function — `rgb()` — which is given three parameters that represent the red, green, and blue channel values of the colors.
- In RGB each channel is represented by a decimal number between 0 and 255

Eg: `background-color: rgb(2, 121, 139);`

RGBA colors — these work in exactly the same way as RGB colors, and so you can use any RGB values. However, there is a fourth value that represents the alpha channel of the color, which controls opacity. If you set this value to **0** it will make the color fully transparent, whereas **1** will make it fully opaque. Values in between give you different levels of transparency.

Eg: `background-color: rgba(197, 93, 161, .7);`

Hexadecimal RGB values

- Each hex value consists of a **hash/pound symbol (#)** followed by six hexadecimal numbers, each of which can take one of 16 values between 0 and f (which represents 15)
— so `0123456789abcdef`.

- Each pair of values represents one of the channels — red, green and blue — and allows us to specify any of the 256 available values for each ($16 \times 16 = 256$).
- These values are a bit more complex and less easy to understand, but they are a lot more versatile than keywords — you can use hex values to represent any color you want to use in your color scheme.

Eg: `background-color: #02798b;`

HSL and HSLA values

- Slightly less well-supported than RGB is the **HSL color model** (not supported on old versions of IE), which was implemented after much interest from designers.
- Instead of red, green, and blue values, the `hsl()` function accepts **hue**, **saturation**, and **lightness** values, which are used to distinguish between the 16.7 million colors, but in a different way.

Hue: The base shade of the color. This takes a value between 0 and 360, representing the angles around a color wheel.

Color wheel

- A color wheel, or color circle, is a chart representing a palette of colors in a circle. A color wheel can display colors identified by two polar coordinates, the angle and the distance from the origin, the circle's center.
- Color wheels are convenient for comparing colors expressed in polar or cylindrical coordinates, like [hsl\(\)](#), [hwb\(\)](#), or [lch\(\)](#).
- In such cases, complementary colors are often found opposite on the same diameter. Similarly, monochromatic colors – colors of the same tone but of different shades – are located on the same radius, and triadic colors – three colors evenly spaced around the color wheel that lead to colors that work well together – are also easy to find.
- Color wheels are used in real life when we want to choose between different hues. For example, when selecting wall paint or the color for a piece furniture.
- In the digital world, color wheels are used in color pickers.



Saturation: How saturated is the color? This takes a value from 0–100%, where 0 is no color (it will appear as a shade of grey), and 100% is full color saturation.

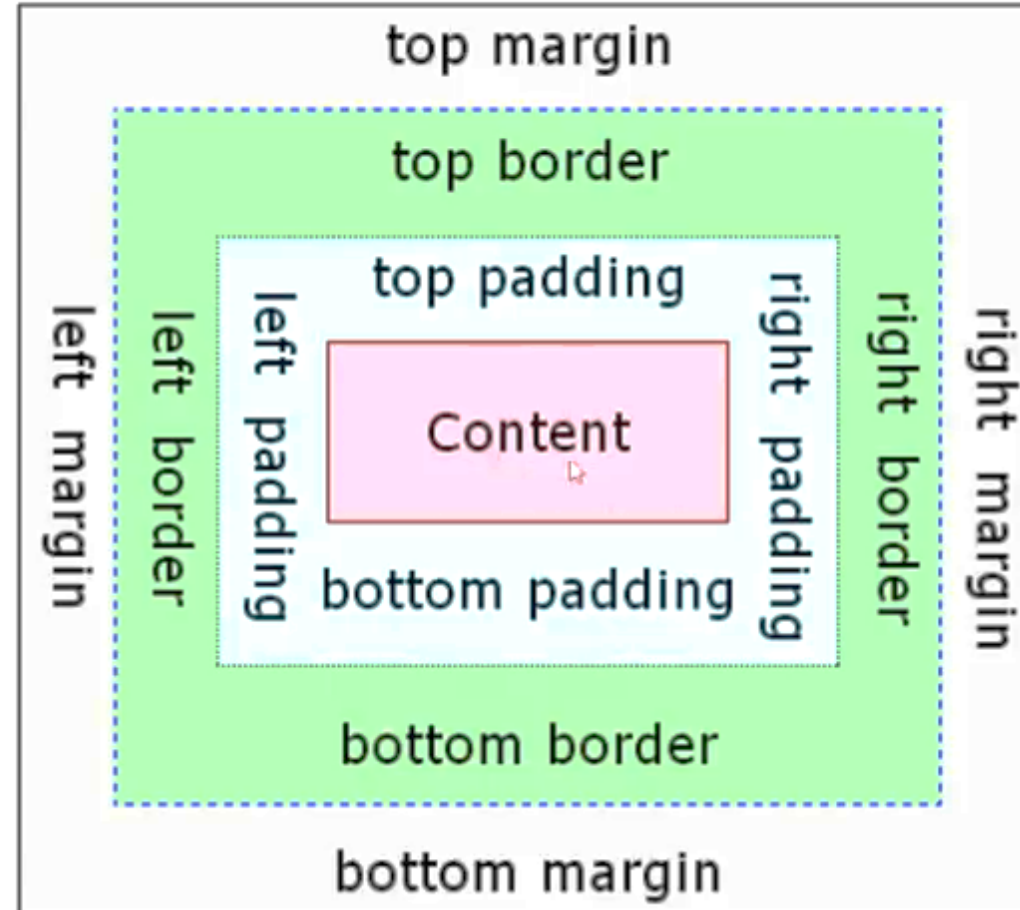
Lightness: How light or bright is the color? This takes a value from 0–100%, where 0 is no light (it will appear completely black) and 100% is full light (it will appear completely white)

Eg: `background-color: hsl(188, 97%, 28%);`

Just as RGB has RGBA, **HSL has an HSLA** equivalent, which gives you the same ability to specify the alpha channel.

Eg: `background-color: hsla(188, 97%, 28%, .3);`

Box Model



Margin & Padding Shorthand

p {

margin-top:5px;

margin-bottom:5px;

margin-right: 10px;

margin-left:10px;

}

p {

margin: 5px 10px 5px 10px;

}

p {

margin: 5px 10px;

}

CSS Border

- The **border** shorthand CSS property sets an element's border.
- It sets the values of [border-width](#), [border-style](#), and [border-color](#).
- The border property may be specified using one, two, or three of the values listed above.
- The order of the values does not matter.
- **The border will be invisible if its style is not defined.** This is because the style defaults to none.

CSS border-width

- The [border-width](#) property specifies the width/thickness of the four borders.
- The width can be set as a specific size (in px, pt, cm, em, etc) or by using one of the three pre-defined values: [thin](#), [medium](#), or [thick](#). Defaults to medium if absent.

Eg: **border-style:** [solid](#); **border-style:** [solid](#);
border-width: [5px](#); **border-width:** [5px 20px](#); (Top/Bottom Lt/Rt)
 border-width: [25px 10px 4px 35px](#); (T/R/B/L)

border-style: [solid](#); **border-style:** [solid](#);
border-width: [thick](#); **border-width:** [medium](#);

CSS border-style

- The Border style property is used to specify the border type which you want to display on the web page.
- There are some border style values which are used with border-style property to define a border.
- Defaults to none if absent.

Value	Description
none	It doesn't define any border.
dotted	It is used to define a dotted border.
dashed	It is used to define a dashed border.
solid	It is used to define a solid border.
double	It defines two borders with the same border-width value.
groove	It defines a 3d grooved border. effect is generated according to border-color value.
ridge	It defines a 3d ridged border. effect is generated according to border-color value.
inset	It defines a 3d inset border. effect is generated according to border-color value.
outset	It defines a 3d outset border. effect is generated according to border-color value.

Eg: border-style: dotted;
border-style: dashed;
border-style: groove;

border-style: solid;
border-style: double;
border-style: ridge;

CSS border-color

The **border-color** property is used to set the color.

- The color can be set by:
 - name - specify a color name, like "red"
 - HEX - specify a HEX value, like "#ff0000"
 - RGB - specify a RGB value, like "rgb(255,0,0)"
 - HSL - specify a HSL value, like "hsl(0, 100%, 50%)"
 - transparent
- The border-color property is not used alone. It is always used with other border properties like "border-style" property to set the border first otherwise it will not work.
- Defaults to currentcolor if absent.

Eg: border-style: solid;
border-color: red;

border-style: solid;
border-color: red green blue yellow;

border-style: solid;
border-color: #ff0000; /red

border-style: solid;
border-color: rgb(255,0,0); /red

border-style: solid;
border-color: hsl(0, 100%, 50%); /red

CSS Rounded Borders (border-radius)

- This CSS property sets the rounded borders and provides the rounded corners around an element, tags, or div. It defines the radius of the corners of an element.
- It gives the rounded shape to the corners of the border of an element. We can specify the border for all four corners of the box in a single declaration using the border-radius. The values of this property can be defined in percentage or length units.

Property	Description
border-top-left-radius	It is used to set the border-radius for the top-left corner
border-top-right-radius	It is used to set the border-radius for the top-right corner
border-bottom-right-radius	It is used to set the border-radius for the bottom-right corner
border-bottom-left-radius	It is used to set the border-radius for the bottom-left corner

- If the bottom-left value is omitted, then it will be same as the top-right. If the value of bottom-right is eliminated, then it will be same as the top-left. Similarly, if top-right is eliminated, then it will be the same as top-left.

Eg: **border-radius: 30px;** **border-radius: 20% 10% ;**
 border-radius: 10% 30% 20% 40%; **border-radius: 50px 50% 50cm 50em;**

CSS Height and Width

- The CSS **height** and **width** properties are used to set the height and width of an element.
- The CSS **max-width** property is used to set the maximum width of an element.
- The height and width properties do not include padding, borders, or margins. It sets the height/width of the area inside the padding, border, and margin of the element.

CSS height and width Values

The **height** and **width** properties may have the following values:

- **auto** - This is default. The browser calculates the height and width
- **length** - Defines the height/width in px, cm, etc.
- **%** - Defines the height/width in percent of the containing block
- **initial** - Sets the height/width to its default value
- **inherit** - The height/width will be inherited from its parent value

height: 100px;
width: 500px;
background-color: blue;

height: 200px;
width: 50%;
background-color: blue;

Positioning in CSS

- CSS helps you to position your HTML element.
- You can put any HTML element at whatever location you like.
- You can specify whether you want the element positioned relative to its natural position in the page or absolute based on its parent element.

- Static
- Relative
- Absolute
- Fixed
- Initial
- Inherit

Syntax

position: static|absolute|fixed|relative|sticky|initial|inherit;

Value	Description
static	Default value. Elements render in order, as they appear in the document flow
absolute	The element is positioned relative to its first positioned (not static) ancestor element
fixed	The element is positioned relative to the browser window
relative	The element is positioned relative to its normal position, so "left:20px" adds 20 pixels to the element's LEFT position
sticky	The element is positioned based on the user's scroll position. A sticky element toggles between relative and fixed, depending on the scroll position. It is positioned relative until a given offset position is met in the viewport - then it "sticks" in place (like position:fixed). Note: Not supported in IE/Edge 15 or earlier.
initial	Sets this property to its default value.
inherit	Inherits this property from its parent element.

Multiple Style Sheets

If some properties have been set for the same selector in different style sheets, the values will be inherited from the more specific style sheet.

For example, an external style sheet has these properties for the h3 selector:

```
h3
{
  color:red;
  text-align:left;
  font-size:8pt;
}
```

And an internal style sheet has these properties for the h3 selector:

```
h3
{
  text-align:right;
  font-size:20pt;
}
```

If the page with the internal style sheet also links to the external style sheet the properties for h3 will be:

```
color:red;
text-align:right;
font-size:20pt;
```

The color is inherited from the external style sheet and the text-alignment and the font-size is replaced by the internal style sheet.

Multiple Styles Will Cascade into One

Styles can be specified:

- inside an HTML element
- inside the head section of an HTML page
- in an external CSS file

Tip: Even multiple external style sheets can be referenced inside a single HTML document.

Cascading order

What style will be used when there is more than one style specified for an HTML element?

Generally speaking we can say that all the styles will "cascade" into a new "virtual" style sheet by the following rules, where number four has the highest priority:

1. Browser default
2. External style sheet
3. Internal style sheet (in the head section)
4. Inline style (inside an HTML element)

So, an inline style (inside an HTML element) has the highest priority, which means that it will override a style defined inside the <head> tag, or in an external style sheet, or in a browser (a default value).

Note: If the link to the external style sheet is placed after the internal style sheet in HTML <head>, the external style sheet will override the internal style sheet!

The Contextual Selector :

There may be situations when you want a given style to be applied to instance of one selector within another. For example, when `<i> ----- </i>` tags occur inside `<p>----</p>` tags, the text color should be green. You can do this by declaring the style tag.

```
<style>
P,i { color:green}
</style>
```

Here only instances of the `<i>` tag within the `<p>` tag will have the style as declared above.

```
<html>
<head> <title> Contextual selector </title>
<style>
    P,i{color:green}
    H1{color:red}
</style>
</head>
<body>
    <H1>Using <i>Selector</i></H1>
    <P>Contextual<i>Selector</i></P>
</body>
</html>
```

In the above code, the `<i>` tag enclosed in the `<P>` tag will have the text in green and the `<i>` tag enclosed in the `<H1>` tag will take the color of `<H1>` tag.

CSS Animation

CSS Animation Properties:

- animation-name
- animation-fill-mode
- animation-duration
- animation-play-state
- animation-timing-function
- animation-delay
- animation-iteration-count
- animation-direction
- animation

```
@keyframes animation-example {  
  from {CSS Properties}  
  to {CSS Properties}  
}
```

```
@keyframes animation-example {  
  0% {CSS Properties}  
  100% {CSS Properties}  
}
```

Property	Description
@keyframes	Specifies the animation code
animation	A shorthand property for setting all the animation properties
animation-delay	Specifies a delay for the start of an animation
animation-direction	Specifies whether an animation should be played forwards, backwards or in alternate cycles
animation-duration	Specifies how long time an animation should take to complete one cycle
animation-fill-mode	Specifies a style for the element when the animation is not playing (before it starts, after it ends, or both)
animation-iteration-count	Specifies the number of times an animation should be played
animation-name	Specifies the name of the @keyframes animation
animation-play-state	Specifies whether the animation is running or paused
animation-timing-function	Specifies the speed curve of the animation

The **animation-direction** property specifies whether an animation should be played forwards, backwards or in alternate cycles.

The animation-direction property can have the following values:

- **normal** - The animation is played as normal (forwards). This is default
- **reverse** - The animation is played in reverse direction (backwards)
- **alternate** - The animation is played forwards first, then backwards
- **alternate-reverse** - The animation is played backwards first, then forwards

The **animation-timing-function** property specifies the speed curve of the animation.

- The animation-timing-function property can have the following values:
- **ease** - Specifies an animation with a slow start, then fast, then end slowly (this is default)
- **linear** - Specifies an animation with the same speed from start to end
- **ease-in** - Specifies an animation with a slow start
- **ease-out** - Specifies an animation with a slow end
- **ease-in-out** - Specifies an animation with a slow start and end
- **cubic-bezier(n,n,n,n)** - Lets you define your own values in a cubic-bezier function

The **animation-fill-mode** property specifies a style for the target element when the animation is not playing (before it starts, after it ends, or both).

The animation-fill-mode property can have the following values:

- **none** - Default value. Animation will not apply any styles to the element before or after it is executing
- **forwards** - The element will retain the style values that is set by the last keyframe (depends on animation-direction and animation-iteration-count)
- **backwards** - The element will get the style values that is set by the first keyframe (depends on animation-direction), and retain this during the animation-delay period
- **both** - The animation will follow the rules for both forwards and backwards, extending the animation properties in both directions

CSS Grid

- CSS Grid is a powerful tool that CSS gives us to easily align items and create two-dimensional layouts.
- Grid and Flexbox are both part of CSS and can both be used in similar ways Flexbox was created first and has bit more browser compatibility.
- I would highly recommend learning both of them.

Grid Containers and Grid Items

The **grid container** is the element that holds **grid items**. Grid items are direct children of grid containers. A container is created with **display:grid**

```
<div class="grid-container">  
  <div>Grid Item 1</div>  
  <div>Grid Item 2</div>  
    
</div>
```

Grid Container Properties

- grid-template columns
- grid-template-rows
- grid-gap
- grid-auto-rows
- grid-auto-columns
- grid-template-areas
- justify-content
- align-items

Grid Item Properties

- grid-column
- grid-row
- grid-area
- align-self
- justify-self

Two Dimensional Layouts

Unlike Flexbox, which is one-dimensional and can be either a row or column, a CSS Grid allows us to create two-dimensional layouts with rows AND columns



CSS Icons

- The simplest way to add an icon to your HTML page, is with an icon library, such as Font Awesome.
- Add the name of the specified icon class to any inline HTML element (like `<i>` or `<span>`).
- All the icons in the icon libraries below, are scalable vectors that can be customized with CSS (size, color, shadow, etc.)

Font Awesome Icons

To use the Font Awesome icons, go to fontawesome.com, sign in, and get a code to add in the `<head>` section of your HTML page:

```
<script src="https://kit.fontawesome.com/yourcode.js" crossorigin =  
"anonymous"></script>
```

Note: No downloading or installation is required!

Example:

```
<!DOCTYPE html>
<html>
<head>
<script src="https://kit.fontawesome.com/a076d05399.js"
crossorigin="anonymous"></script>
</head>
<body>
  <i class="fas fa-cloud"></i>
  <i class="fas fa-heart"></i>
  <i class="fas fa-car"></i>
  <i class="fas fa-file"></i>
  <i class="fas fa-bars"></i>
</body>
</html>
```

Result:



Google Icons

To use the Google icons, add the following line inside the <head> section of your HTML page:

```
<link rel="stylesheet" href="https://fonts.googleapis.com/icon?family=Material+Icons">
```

Note: No downloading or installation is required!

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="https://fonts.googleapis.com/icon?family=Material+Icons">
</head>
<body>
  <i class="material-icons">cloud</i>
  <i class="material-icons">favorite</i>
  <i class="material-icons">attachment</i>
  <i class="material-icons">computer</i>
  <i class="material-icons">traffic</i>
</body>
</html>
```

Result:



CSS Variables - The var() Function

- CSS variables can be create variables with local or global scope. We can change the variables with JavaScript, and change the variables based on media queries.
- A good way to use CSS variables is when it comes to the colors of your design. Instead of copy and paste the same colors over and over again, you can place them in variables.

The **var()** function is used to insert the value of a CSS variable.

Syntax: **var(--name, value)**

Value	Description
<i>name</i>	Required. The variable name (must start with two dashes)
<i>value</i>	Optional. The fallback value (used if the variable is not found)

Note: The variable name must begin with two dashes (--) and it is case sensitive!

Points to Note:

- **CSS variables** can have a global or local scope.
- **Global variables** can be accessed/used through the entire document, while local variables can be used only inside the selector where it is declared.
- To create a variable with **global scope**, declare it inside the **:root** selector. The **:root** selector matches the document's root element.
- To create a variable with local scope, declare it inside the selector that is going to use it.
- The **:root** class is used to define the variable in CSS.
- A variable is a **data container** that stores data, and the **root** class defines it.
- You can store any **value** of **CSS property** in a variable.